

Sortiervverfahren

Programmieren und Software-Engineering Theorie

17. März 2026

- Wir betrachten einfache Sortialgorithmen als Beispiele für Algorithmen
- Ziel: Sortiere die Zahlen in einem Array (aufsteigend)

Array mit 10 Integer-Elementen

9	4	12	1	7	3	10	5	2	8
0	1	2	3	4	5	6	7	8	9

Abbildung: Array mit 10 Elementen (Index unter den Zellen).

Bubblesort

Algorithmus 1: Bubblesort

Input: Array a

Result: Sortiertes Array a

```
1 Function BUBBLESORT( $a$ )
2   for  $i = 0; i < a.length - 1; i++$  do
3      $swapped = \text{false}$ 
4     for  $j = 0; j < a.length - 1 - i; j++$ 
5       do
6         if  $a[j] > a[j + 1]$  then
7           SWAP( $a, j, j + 1$ )
8            $swapped = \text{true}$ 
9     if not  $swapped$  then
10      Break
```

Bubblesort (Beispiel)

Durchlauf i	Array-Zustand
Initial	9 4 12 1 7 3 10 5 2 8
0	4 9 1 7 3 10 5 2 8 12
1	4 1 7 3 9 5 2 8 10 12
2	1 4 3 7 5 2 8 9 10 12
3	1 3 4 5 2 7 8 9 10 12
4	1 3 4 2 5 7 8 9 10 12
5	1 3 4 2 5 7 8 9 10 12
6	1 3 2 4 5 7 8 9 10 12
7	1 2 3 4 5 7 8 9 10 12
8	1 2 3 4 5 7 8 9 10 12 (sortiert)

Insertionsort

Algorithmus 2: Insertionsort

Input: Array a

Result: Sortiertes Array a

```
1 Function INSERTIONSORT( $a$ )
2    $i = 1$ 
3   while  $i < a.length$  do
4      $j = i$ 
5     while  $j > 0 \wedge a[j - 1] > a[j]$  do
6       SWAP( $a, j, j-1$ )
7        $j = j - 1$ 
8    $i = i + 1$ 
```

Insertionsort (Beispiel)

i (nächster Index)	Array-Zustand (linker Teil sortiert)
Initial	9 4 12 1 7 3 10 5 2 8
1	4 9 12 1 7 3 10 5 2 8
2	4 9 12 1 7 3 10 5 2 8
3	1 4 9 12 7 3 10 5 2 8
4	1 4 7 9 12 3 10 5 2 8
5	1 3 4 7 9 12 10 5 2 8
6	1 3 4 7 9 10 12 5 2 8
7	1 3 4 5 7 9 10 12 2 8
8	1 2 3 4 5 7 9 10 12 8
9	1 2 3 4 5 7 8 9 10 12 (sortiert)

Selectionsort

Algorithmus 3: Selectionsort

Input: Array a

Result: Sortiertes Array a

```
1 Function SELECTIONSORT( $a$ )
2   for  $i = 0; i < a.length - 1; i++$  do
3      $jMin = i$ 
4     for  $j = i + 1; j < a.length; j++$  do
5       if  $a[j] < a[jMin]$  then
6          $jMin = j$ 
7     if  $jMin \neq i$  then
8       SWAP( $a, i, jMin$ )
```

Selectionsort

i	Array-Zustand
Initial	9 4 12 1 7 3 10 5 2 8
0	1 4 12 9 7 3 10 5 2 8
1	1 2 12 9 7 3 10 5 4 8
2	1 2 3 9 7 12 10 5 4 8
3	1 2 3 4 7 12 10 5 9 8
4	1 2 3 4 5 12 10 7 9 8
5	1 2 3 4 5 7 10 12 9 8
6	1 2 3 4 5 7 8 12 9 10
7	1 2 3 4 5 7 8 9 12 10
8	1 2 3 4 5 7 8 9 10 12 (sortiert)

Laufzeitanalyse

Worst-Case Analyse einfacher Sortieralgorithmen bei n Elementen:

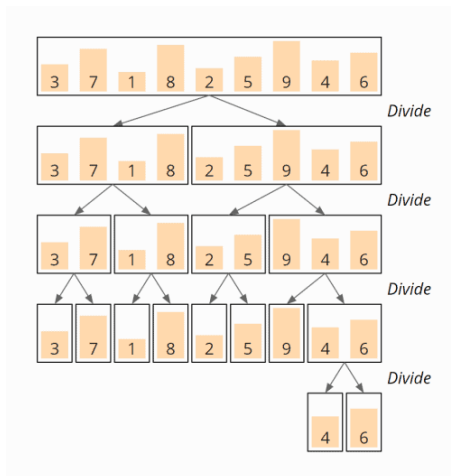
Algorithmus	Vergleiche	Swaps	Gesamt
Bubblesort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertionsort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Selectionsort	$O(n^2)$	$O(n)$	$O(n^2)$

Die Gesamtlaufzeit von $O(n^2)$ ist in der Praxis problematisch, d.h. zu langsam. Daher kommen üblicherweise bessere Algorithmen zum Einsatz, die eine Gesamtlaufzeit von $O(n \log n)$ aufweisen.

Mergesort

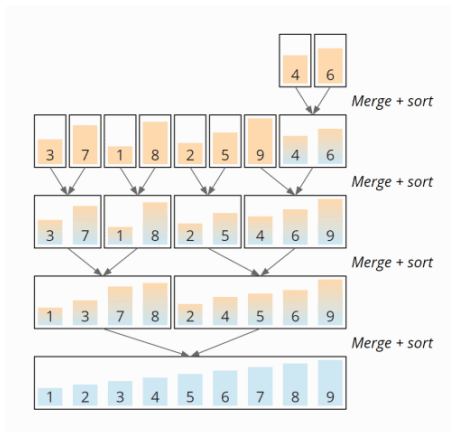
- Der Algorithmus *Mergesort* sortiert die Daten nach dem Prinzip **Divide & Conquer** (dt.: Teile und Herrsche)
- Vorgehensweise bei Divide & Conquer:
 - *Teile* das Problem in kleinere Teilprobleme
 - *Löse* diese Teilprobleme
 - Füge Teillösungen zusammen
- **Mergesort**
 - Zahlenfolge (Array) wird durch rekursive Aufrufe unterteilt.
 - Die Sortierung wird beim anschließenden Zusammenfügen der Arrays erreicht
 - Mergesort weist bessere Laufzeit-Eigenschaften als die bisher besprochenen Sortialgorithmen auf!

Mergesort



Quelle: <https://www.happycoders.eu/de/algorithmen/mergesort/>

Mergesort



Quelle: <https://www.happycoders.eu/de/algorithmen/mergesort/>

Mergesort

- Rekursive Mergesort-Implementierung
- Aufruf mit $\text{MERGESORT}(A, 0, \text{len}(A))$

Algorithmus 4: Mergesort

Input: Array $A[l..r]$

```
1 Function MERGESORT( $A, l, r$ )
2   if  $l = r$  then
3     return  $[A[l]]$ 
4    $m = l + \lfloor (r - l) / 2 \rfloor$ 
5    $L = \text{MERGESORT}(A, l, m)$ 
6    $R = \text{MERGESORT}(A, m + 1, r)$ 
7   return  $\text{MERGE}(L, R)$ 
```

Merge-Methode

Algorithmus 5: Merge zweier sortierter Listen

Input: sortierte Arrays $L[0..a-1]$, $R[0..b-1]$

Output: zusammengefügt sortiertes Array $S[0..a+b-1]$

```
1 Function MERGE( $L, R$ )
2    $leftPos = 0$ 
3    $rightPos = 0$ 
4    $targetPos = 0$ 
5   while  $leftPos < a$  and  $rightPos < b$  do
6     if  $L[leftPos] \leq R[rightPos]$  then
7        $S[targetPos] = L[leftPos]$ 
8        $leftPos = leftPos + 1$ 
9     else
10       $S[targetPos] = R[rightPos]$ 
11       $rightPos = rightPos + 1$ 
12     $targetPos = targetPos + 1$ 
13  while  $leftPos < a$  do
14     $S[targetPos] = L[leftPos]$ 
15     $leftPos = leftPos + 1$ 
16     $targetPos = targetPos + 1$ 
17  while  $rightPos < b$  do
18     $S[targetPos] = R[rightPos]$ 
19     $rightPos = rightPos + 1$ 
20     $targetPos = targetPos + 1$ 
21  return  $S$ 
```

Quicksort

- Effizienter Sortialgorithmus von Tony Hoare (1959)
- Ebenso Divide & Conquer
- Array wird anhand von *Pivot-Element* rekursiv geteilt.
- Letztes Element im Teil-Array ist Pivot-Element¹.
- Trotz Worst-Case-Laufzeit von $O(n^2)$ in der Praxis schneller als Mergesort.
- Der Average-Case $O(n \log n)$ tritt so gut wie immer ein!

¹Es gibt viele Varianten zur Wahl des Pivot-Elements.

Quicksort

Algorithmus 6: Quicksort

Input: Array $A[l..r]$

```
1 Function QUICKSORT( $A, l, r$ )  
2   if  $l \geq r$  then  
3     return  
4    $p = \text{PARTITION}(A, l, r)$   
5   QUICKSORT( $A, l, p - 1$ )  
6   QUICKSORT( $A, p + 1, r$ )
```

Algorithmus 7: Partition

Input: Array $A[l..r]$

```
1 Function PARTITION( $A, l, r$ )
2    $pivot = A[r]$ 
3    $i = l$ 
4    $j = r - 1$ 
5   while  $i < j$  do
6     while  $A[i] < pivot$  do
7        $i = i + 1$ 
8     while  $j > l$  and  $A[j] \geq pivot$  do
9        $j = j - 1$ 
10    if  $i < j$  then
11      SWAP( $A, i, j$ )
12       $i = i + 1$ 
13       $j = j - 1$ 
14    if  $i = j$  and  $A[j] < pivot$  then
15       $i = i + 1$ 
16    if  $A[i] \neq pivot$  then
17      SWAP( $A, i, r$ )
18    return  $i$                                 // finale Pivot-Position
```

Quicksort (Beispiel)

Ablauf Zahlenfolge 3, 7, 1, 8, 2, 5, 9, 4, 6:

|> 3 7 1 8 2 5 9 4 6 <|

|> 3 7 1 8 2 5 9 4 (6)<|

|> 3 4 1 8 2 5 9 7 6 <|

|> 3 4 1 5 2 8 9 7 6 <|

|> 3 4 1 5 2 [6] 9 7 8 <|

|> 3 4 1 5 (2)<| 6 9 7 8

|> 1 4 3 5 2 <| 6 9 7 8

|> 1 [2] 3 5 4 <| 6 9 7 8

1 2 |> 3 5 (4)<| 6 9 7 8

1 2 |> 3 [4] 5 <| 6 9 7 8

1 2 3 4 5 6 |> 9 7 (8)<|

1 2 3 4 5 6 |> 7 9 8 <|

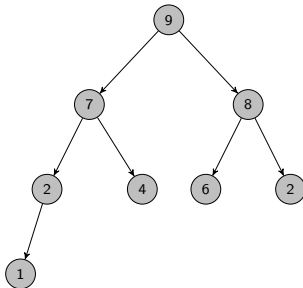
1 2 3 4 5 6 |> 7 [8] 9 <|

Anmerkung: |> und <| notieren die Grenzen (*l* und *r*). () notiert die Wahl des

Pivot-Elements, [] die Platzierung an der endgültigen Stelle.

Heap

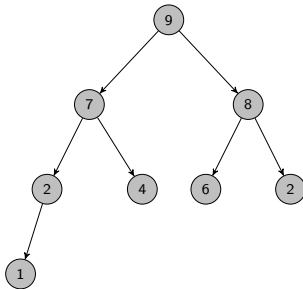
- (Max-)Heap: Vollständiger Binärbaum mit der Eigenschaft: Wert jedes Knotens größer oder gleich seiner Kinder.
- Max-Heap mit 8 Elementen: Die Werte in den Knoten stellen die gespeicherten Werte dar:



Da Baumstruktur implizit gegeben, können die Elemente (Ebene für Ebene betrachte) in einem Array $[9, 7, 8, 2, 4, 6, 2, 1]$ gespeichert werden.

Heap

- (Max-)Heap: Vollständiger Binärbaum mit der Eigenschaft: Wert jedes Knotens größer oder gleich seiner Kinder.
- Max-Heap mit 8 Elementen: Die Werte in den Knoten stellen die gespeicherten Werte dar:

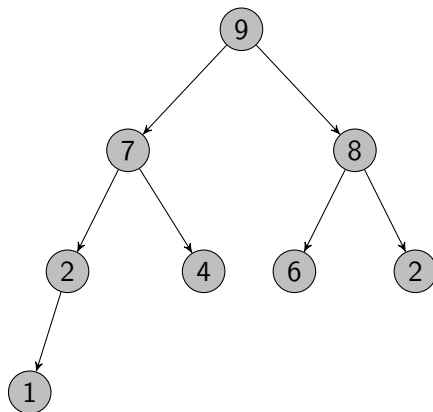


Da Baumstruktur implizit gegeben, können die Elemente (Ebene für Ebene betrachte) in einem Array $[(9), (7, 8), (2, 4, 6, 2), 1]$ gespeichert werden.

Heapsort

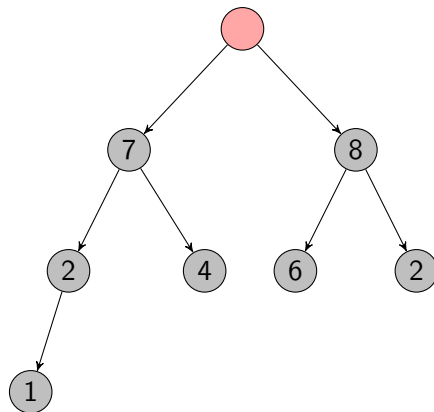
- Nutzen für Sortierung: teilweise Ordnung
- Start mit gültigem Heap (kann in $O(n)$ aufgebaut werden)
- Entnahme Maximum: $O(1)$
- Platzierung des Maximums am Ende, und Verkleinerung des Heaps um ein Element
- Getaushtes Element von letzter Stelle ist nun in Wurzel
- → Versickern/Heapify

Versickern/Heapify



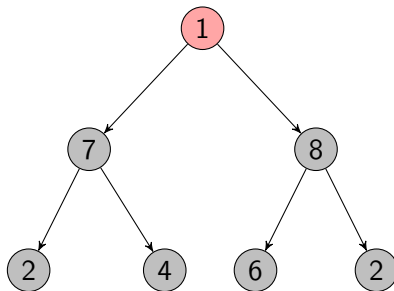
Initialer Heap: Maximum steht in Wurzel [9, 7, 8, 2, 4, 6, 2, 1]

Versickern/Heapify



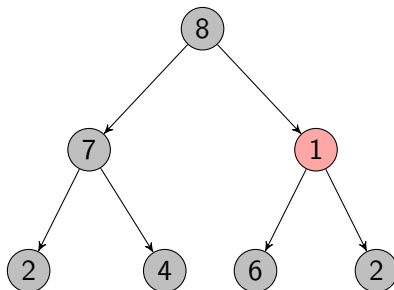
Entnahme der Wurzel (Platzierung am Ende des Heaps)
[1, 7, 8, 2, 4, 6, 2 | 9]

Versickern/Heapify



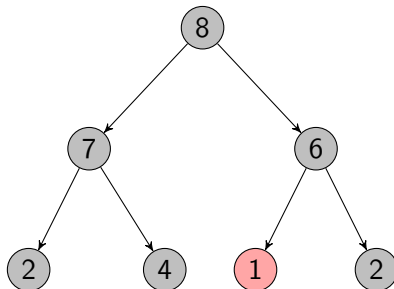
Entnahme der Wurzel (Platzierung am Ende des Heaps)
[1, 7, 8, 2, 4, 6, 2 | 9]

Versickern/Heapify



Versickern von 1 durch Tausch mit größtem Kind (8)

Versickern/Heapify



Weiter-versickern von 1 durch Tausch mit größtem Kind (8)

Aufbau des initialen Heaps

- Bottom-Up-Aufbau eines Max-Heaps aus $[1, 9, 8, 7, 4, 6, 2, 2]$.
- Versickern aller Nicht-Blätter

Arrayzustand	Kommentar
1, 9, 8, 7, 4, 6, 2, 2	Ausgangsfolge (unsortierte Anordnung)
	Heapify bei Index 3 (7 mit Kind 2): alles ok.
	Index 2 (8 mit Kindern 6 und 2): alles ok.
	Index 1 (9 mit Kindern 7 und 4): $9 \geq \max(7,4)$, bleibt.
9, 1, 8, 7, 4, 6, 2, 2	Index 0 (1 mit Kindern 9 und 8): $\rightarrow$ tausche mit 9
9, 7, 8, 1, 4, 6, 2, 2	Jetzt 1 an Index 1 (Kinder 7,4) $\rightarrow$ tausche mit 7
9, 7, 8, 2, 4, 6, 2, 1	Jetzt 1 an Index 3 (Kind 2) $\rightarrow 1 < 2 \rightarrow$ tausche
9, 7, 8, 2, 4, 6, 2, 1	Fertiger Max-Heap

Ausführung von Heapsort

Detaillierte Zwischenschritte von Heapsort mit der Eingabefolge [9, 7, 8, 2, 4, 6, 2, 1]

Schritt	Arrayzustand	Kommentar
1	1, 7, 8, 2, 4, 6, 2 9 8, 7, 6, 2, 4, 1, 2 9	Vertausche Wurzel 9 mit letztem Heap-Element 1 Heapify bei Index 0: versickern von 1
2	2, 7, 6, 2, 4, 1 8, 9 7, 4, 6, 2, 2, 1 8, 9	Vertausche Wurzel 8 mit letztem Heap-Element 2 Heapify bei Index 0: versickern von 2
3	1, 4, 6, 2, 2 7, 8, 9 6, 4, 1, 2, 2 7, 8, 9	Vertausche Wurzel 7 mit letztem Heap-Element 1 Heapify bei Index 0: versickern von 1
4	2, 4, 1, 2 6, 7, 8, 9 4, 2, 1, 2 6, 7, 8, 9	Vertausche Wurzel 6 mit letztem Heap-Element 2 Heapify bei Index 0: versickern von 2
5	2, 2, 1 4, 6, 7, 8, 9 2, 2, 1 4, 6, 7, 8, 9	Vertausche Wurzel 4 mit letztem Heap-Element 2 Heapify bei Index 0: keine Veränderung nötig
6	1, 2 2, 4, 6, 7, 8, 9 2, 1 2, 4, 6, 7, 8, 9	Vertausche Wurzel 2 mit letztem Heap-Element 1 Heapify bei Index 0: versickern von 1
7	1 2, 2, 4, 6, 7, 8, 9 1 2, 2, 4, 6, 7, 8, 9	Vertausche Wurzel 2 mit letztem Heap-Element 1 Heapify nicht nötig
–	1, 2, 2, 4, 6, 7, 8, 9	Endergebnis: Array aufsteigend sortiert

Definition

Worst-Case Analyse einfacher Sortieralgorithmen:

Algorithmus	Vergleiche	Swaps
Bubblesort	$O(n^2)$	$O(n^2)$
Insertionsort	$O(n^2)$	$O(n^2)$
Selectionsort	$O(n^2)$	$O(n)$

Analyse weiterer Algorithmen:

Algorithmus	Worst-Case	Average-Case
Heapsort	$O(n \log n)$	$O(n \log n)$
Mergesort	$O(n \log n)$	$O(n \log n)$
Quicksort	$O(n^2)$	$O(n \log n)$

Quicksort:

- Worst-Case bei Quicksort wird nur selten erreicht
- In der Praxis schneller als Mergesort und Heapsort

Definition

In Java verwendet die Standardbibliothek verschiedene Sortiervverfahren: Für primitive Arrays (z.B. `int[]` oder `double[]`) setzt die JDK-Implementierung seit Java 7 einen Dual-Pivot-Quicksort ein (leistungsfähig, jedoch instabil). Für Objekt-Arrays und für `List`-Implementierungen kommt eine stabile, adaptive Sortierung (Timsort) zum Einsatz.

Timsort ist ein stabiler, adaptiver Hybridalgorithmus, der bereits sortierte Teilfolgen (Runs) erkennt und durch Kombination von Insertionsort und einem effizienten Merge-Verfahren praktisch meist in $O(n \log n)$ Zeit sortiert.

Zusätzlich bietet `Arrays.parallelSort()` parallele Varianten (Fork/Join-basiert), die bei großen Datenmengen die Performance verbessern können.